192526027b

Experiment 1 : 8-Puzzle problem

Aim :

To write a python program to solve the 8-puzzle problem using the A* search algorithm and manhattan distance heuristic

Algorithm :

1) start the program and initialize the start and goal states.

2) Calculate the manhattan distance using a heuristic function.

3) Insert the start state into the priority queue.

4) Select the state with lowest cost

$$f(n) = g(n) + h(n)$$

5) Generate new states by moving the empty tile in four directions.

6) Repeat until the goal state is reached.

7) Print the solution path and stop

Program: 1925260276

```
import heapq

def h(s,g):
    return sum (abs (s.index(i)//3 - g.index(i)//3 ) +
                (abs (s.index(i)%.3 - g.index(i)%.3 )
                for i in range (1,9))

def astar (start, goal):
    q = [[h(start, goal), 0, start, []]]
    seen = set()
    while q:
        f, cost, s, path = heapq.heappop(q)
        if s == goal:
            return path + [s]
        if tuple(s) in seen:
            continue
        seen.add (tuple(s))
        i = s.index (0)
        x, y = divmod (i,3)
        for dx, dy in [(1,0), (-1,0), (0,1), (0,-1)]:
            nx, ny = x+dx, y+dy
            if 0 <= nx < 3 and 0 <= ny < 3:
                t = s[:]
                j = nx*3 + ny
                t[i], t[j] = t[j], t[i]
```

Output :

(1, 2, 3)

(4, 5, 6)

(7, 8, 9)

(1, 2, 3)

(4, 5, 6)

(7, 8, 0)

Goal reached !

Experiment : 2 (Queen Problem) 192526076

Aim:

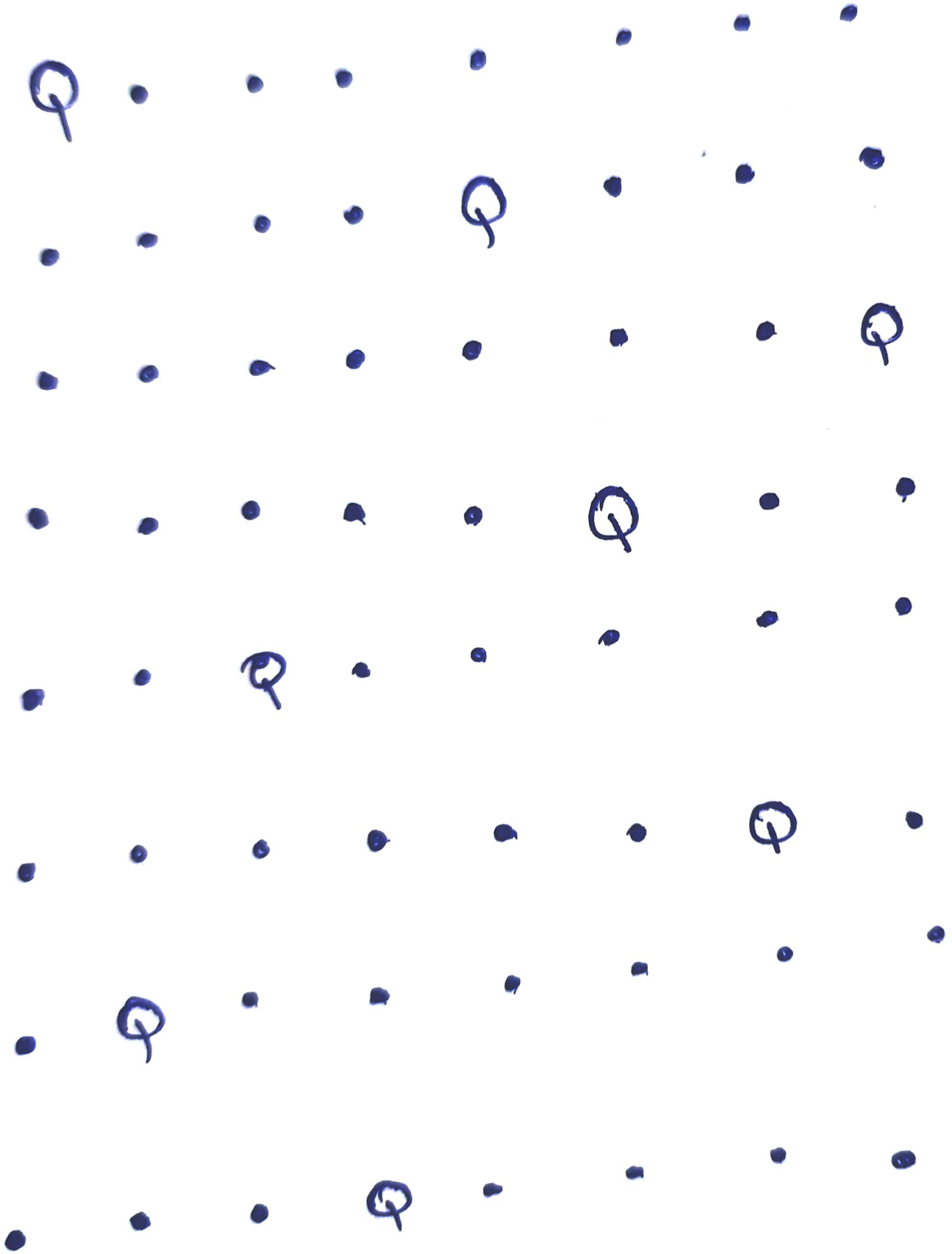
To write a python program to solve the problem using the backtracking method.

Algorithm:

1) Initialize an empty chessboard
2) Place one queen in each row
3) check whether the queen's column and diagonals are safe.
4) If safe, place the queen's column and diagonals are safe.
5) If no position is safe, backtrack
6) Print the solution when all 8 queens are placed.

```
if tuple(t) not in seen class:                    192526027 6
    g = cost + 1
    heapq.heappush(q, [g+h(t, goal), g, t, path+[s]])
return None
start = [1, 2, 3, 4, 0, 5, 6, 7, 8]
goal  = [1, 2, 3, 4, 5, 6, 7, 8, 0]
for s in astar(start, goal) or []:
    print(s[:3], s[3:6], s[6:])
```

Output :



Experiment 3 : Water jug problem

1925260276

Aim :

To write a python program to solve the water jug problem using Breadth-First search (BFS).

Algorithm :

1) Intialize the both jugs as empty
2) Create a queue and set to store visited states.
3) Remove a state from the queue and check whether the target amount is reached.
4) Generate new states by fillings, emptying or transferring water between jugs.
5) Repeat until the target amount is reached.
6) Print the solution.

192526027 6

Program:

```
from collections import deque

def water-jug(a, b, target):
    visited = set()
    q = deque([[0,0]])

    while q:
        x, y = q.popleft()

        if x == target or y == target:
            print("solution:", (x,y))
            return

        if (x, y) in visited:
            continue
        visited.add((x, y))

        q.extend([
            (a,y), (x, b), (0, y), (x, 0),
            (min(x+y, a), max(0, x+y-a)),
            (max(0, x+y-b), min(x+y, b))
        ])

J)

water-jug(4, 3, 2)
```

Output :

(0, 0)

(0, 3)

(3, 0)

(3, 3)

(4, 2)

Goal reached !

Experiment 4: Crypt-Arithmetic Problem

1925260276

Aim:

To write a python program to solve the crypt arithmetic problem SEND + MORE = MONEY using permutations.

Algorithm:

1) Import the itertools module
2) Assign unique digits to the letters.
3) Ensure that S and M are not zero.
4) Convert the words into numbers
5) check whether SEND + MORE = MONEY
6) Print the solution and stop.

Program:

192526027b

```
import itertools

def solve():
    letters = "SENDMORY"

    for perm in itertools.permutations(range(10),
    len(letters)):

        s, e, n, d, m, o, r, y = perm

        if s == 0 or m == 0:
            continue

        send = s*1000 + e*100 + n*10 + d

        more = m*1000 + o*100 + r*10 + e

        money = m*10000 + o*1000 + n*100 + e*10 + y

        if send + more == money:
            print("solution:", send, "+", more, "=",
            money)

            return

solve()
```

Output :

SEND = 9567

MORE = 1085

MONEY = 10652

Experiment 6: Missionaries and Canibals Problem

192526076

Aim:

To write a python program to solve the missionaries and Cannibals Problem using Breadth first search (BFS)

Algorithm:

1) Intialize three missionaries and three cannibals on starting bank.

2) Create a queue to store states and a set to track visited states.

3) Select a state and check whether the goal is reached

4) Generate the possible boat movements carrying one or two people.

5) check that missionaries are never outnumber by canibals on either bank when missionari are present.

6) Repeat until the goal state is reached

7) Print the result

Program: 192526027

```
from collection import deque

def valid (m, c):
    return (m == 0 or m >= c) and (3-m == 0 or
    3-m >= 3-c)

def missionaries - cannibals ():
    start = (3, 3, 3)
    goal = (0, 0, 0)
    q = deque ( [start] )
    visited = set()

while q:
    m, c, b = q.popleft()

if (m, c, b) == goal:
    Print ("solved!")
    return

if (m, c, b) in visited:
    continue
```

Output:

(3, 3, 6)

(3, 1, 1)

(3, 2, 0)

(3, 0, 1)

(3, 1, 0)

(1, 1, 1)

(2, 2, 0)

(0, 2, 1)

(0, 3, 0)

(0, 1, 1)

(0, 2, 0)

(0, 0, 1)

All crossed safely!